

CPS 200

Lecture 07

Feb 2, 2026

Kyu Hur

From Lecture 06

Return Values

- What is the difference between `print()` and `return`?
- What is a **fruitful** function? What is a **void** function? Can you give examples?

Return Values

- Can you write a function named `multiply_by_3()` that takes a parameter `num`, multiplies it by 3, then returns the value?

```
def multiply_by_3(num):  
    return 3 * num
```

```
val = multiply_by_3(10)  
print(val)
```

Return Values

- What is the value of **val** on the left vs the right?

```
def multiply_by_3(num):  
    return num * 3
```

```
val = 3  
val = multiply_by_3(val)  
print(val)
```

```
def multiply_by_3(num):  
    print(num * 3)
```

```
val = 3  
multiply_by_3(val)  
print(val)
```

Return Values and Conditionals

- What is the value of x below? Discuss with partners

```
def func(num):  
    value = 0  
    for i in range(num, -1, -1):  
        if i % 2 == 0 and i % 3 == 0:  
            value = value + i  
    return value  
  
x = func(20)
```

Recursion + Return

- What is the output of the following code? Discuss with partners

```
def fibonacci(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibonacci(n-1) + fibonacci(n-2)  
  
num = fibonacci(5)  
print(num)
```

`isinstance()`

- What does `isinstance()` do? How is it different from `type()`? Try to guess the output of the following with partners:
 - `isinstance(10, int)` **True**
 - `isinstance("hello", str)` **True**
 - `isinstance(10.0, int)` **False**
 - `type(10)` **int**
 - `type("hello")` **str**
 - `type(10.0)` **float**

`isinstance()` **and** `not`

- Because `isinstance()` is a boolean, it's commonly used as a conditional
- `not` is a keyword that outputs the opposite of the boolean expression
e.g.
 - `not True` **False**
 - `not False` **True**
 - `not (10 % 2 == 0)` **False**
 - `not (10 % 3 == 0)` **True**

`isinstance()` **and** not

- What is the output of the following code?
 - `isinstance("string", str)` **True**
 - `not isinstance("string", str)` **False**
 - `isinstance(9, int)` **True**
 - `not isinstance(9, int)` **False**

Lecture 07

for loops in str types

- Previously, we ran for loops with integers only
- The following code would go from 0 to 10 (exclusive)—printing 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

```
for i in range(10):  
    print(i)
```

for loops in str types

- `range(10)` essentially creates a list from 0 to 9 on the fly
- The code blocks below are virtually the same

```
for i in range(10):  
    print(i)
```

```
for i in [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]:  
    print(i)
```

for loops in str types

- So why don't we simply just use that “list” of numbers in all functions?
- If we want to loop over a big value e.g. 1_000_000
- It's more practical to use the `range()` function

```
for i in range(1_000_001):  
    print(i)
```

```
for i in [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ..., 1_000_000]:  
    print(i)
```

for loops in str types

- We can also run for loops through strings
- Below is a for loop through the string value "Gadsby", where *i* represents the character
- What do you think gets printed?

```
for i in "Gadsby":  
    print(i)
```



```
"G"  
"a"  
"d"  
"s"  
"b"  
"y"
```

for loops in str types

- Think of it as though you are loop through a “list” of characters (which is what a string is) e.g.

```
for i in ["G", "a", "d", "s", "b", "y"]:  
    print(i)
```


for loops in str types

- Frequently, we also rename the iterator variable to something that is more recognizable e.g.

```
for let in "Gadsby":  
    print(let)
```

or

```
for letter in "Gadsby":  
    print(letter)
```

for loops in str types

- Renaming the iterator variable has no effect on the behavior of the code, but it is good to name variables as close to the value it represents

```
for let in "Gadsby":  
    print(let)
```

or

```
for letter in "Gadsby":  
    print(letter)
```

for loops in str types

- What is the output of the following code? Discuss with partners.

```
for let in "Gadsby":  
    if let == "s":  
        print(let)
```

"s"

for loops in str types

- What is the output of the following code? Discuss with partners.

```
for let in "lakeland university japan":  
    if let == "lakeland":  
        print(let)
```

This is a trap, this prints nothing

for loops in str types

- What is the output of the following code? Discuss with partners.

```
times_a = 0
for let in "lakeland university japan":
    if let == "a":
        times_a = times_a + 1

print(times_a)    4
```

Think Python

Think Python: Iteration and Search

- [Chapter 07.ipynb](#)

End